

TINKUY

Plataforma de eventos universitarios

Informe del proyecto · Aplicación Django

Evidencia del funcionamiento y conceptos clave de Django aplicados

Autor: Santiago Narváez

Materia: Programación II

Universidad UTE · Período 2026-01

Stack: Django 5.2 · PostgreSQL · Tailwind CSS

Repositorio: github.com/czambrano1997/ProgII_UTE202601

Fecha: 20 de junio de 2026

Tabla de contenido

Nota: Para rellenar el índice, abre el documento en Word, haz clic derecho sobre la tabla y elige «Actualizar campos» (F9).

1. Introducción

Tinkuy es una aplicación web desarrollada con Django para gestionar **eventos y conferencias universitarias**: permite publicar eventos, organizar sus sesiones y salas, registrar a ponentes y asistentes, generar inscripciones y producir pósteres de difusión. El nombre proviene del kichwa **tinkuy** («encuentro»), en alusión a la confluencia de personas en un evento.

Este informe documenta el **funcionamiento de la aplicación terminada** mediante capturas de pantalla y, al mismo tiempo, explica los **conceptos clave de Django** que cada pantalla demuestra. Cada sección del recorrido incluye un espacio para la captura correspondiente y un recuadro «Conceptos Django demostrados» que conecta lo que se ve con el código que lo produce.

1.1 Pila tecnológica

Componente	Tecnología	Rol
Framework	Django 5.2	ORM, vistas, plantillas, administración y enrutamiento.
Base de datos	PostgreSQL	Persistencia y consultas SQL avanzadas para reportes.
Estilos	Tailwind CSS	Interfaz mediante utilidades, compilada con su CLI.
Lenguaje / herramientas	Python 3.13, uv, pyright	Gestión de dependencias y verificación estricta de tipos.

Tabla 1. Pila tecnológica.

2. Arquitectura general

La aplicación se organiza en **capas** con responsabilidades separadas. Las vistas no acceden directamente a la base de datos: siempre piden los datos a la **capa de repositorios**, que es la única que habla con PostgreSQL. Las plantillas se componen a partir de **componentes** reutilizables. El flujo de una petición es siempre el mismo:

```
Petición HTTP
-> urls.py      (enrutamiento a la vista)
-> vista (FBV)  (orquesta la respuesta)
-> repositorio  (ORM o SQL parametrizado)
-> modelos / Postgres (datos)
-> plantilla + componentes (HTML)
-> Respuesta HTTP
```

Figura 1. Flujo petición–respuesta por capas.

El proyecto se construyó de forma **incremental**; el historial de commits refleja cada capa terminada por separado:

Commit	Aporte
chore: project skeleton, tooling and strict typing	Esqueleto, herramientas y tipado estricto.
feat: domain models and migrations	Los modelos del dominio y sus migraciones.

Commit	Aporte
feat: SQL repositories (ORM + raw SQL reports)	Capa de datos: ORM y reportes en SQL crudo.
feat: views and reusable UI component structure	Vistas y componentes de interfaz reutilizables.
feat: Django admin for every model	Administración para todos los modelos.
feat: HTML event and lineup posters	Pósteres de evento y de cartelera.
chore: tests, pyright gate, ADRs and Makefile	Pruebas, verja de tipos y documentación.
ui theme update / project cleanup	Ajustes finales de tema y limpieza.

Tabla 2. Historial de desarrollo (resumen de commits).

3. Modelo de datos

El dominio se define con **ocho modelos** en `events/models.py`. En Django, una clase de modelo es una tabla, cada atributo es una columna y cada instancia es una fila; el **ORM** traduce todo esto a SQL. Cada modelo tiene un `__str__` tipado y una clase `Meta` con orden y nombres en español (`verbose_name`).

Modelo	Rol	Relaciones
Event	Evento o congreso.	1:N → Session
Room	Sala física.	1:N → Session (PROTECT)
Speaker	Ponente.	N:M ↔ Session; 1:1 → SpeakerProfile
SpeakerProfile	Datos extendidos del ponente.	1:1 → Speaker
Session	Charla dentro de un evento.	FK Event/Room; N:M Speaker
Attendee	Asistente.	1:N → Registration
Registration	Inscripción asistente-sesión.	FK Attendee/Session
SiteConfiguration	Configuración (marca, navegación).	—

Tabla 3. Los ocho modelos del dominio.

3.1 Tipos de campo y relaciones

Los modelos ejercen una amplia variedad de **tipos de campo** y los tres tipos de **relación**. El siguiente extracto de `Session` muestra las tres relaciones y el uso de `choices`:

```
class Level(models.TextChoices):
    BEGINNER = "beginner", "Principiante"
    INTERMEDIATE = "intermediate", "Intermedio"
    ADVANCED = "advanced", "Avanzado"

class Session(models.Model):
    event = models.ForeignKey(Event, on_delete=models.CASCADE, related_name="sessions")
    room = models.ForeignKey(Room, on_delete=models.PROTECT, related_name="sessions")
    speakers = models.ManyToManyField(Speaker, related_name="sessions")
    level = models.CharField(max_length=20, choices=Level.choices)
    duration = models.DurationField()
    ...
    class Meta:
        unique_together = [("event", "slug")]
```

Código 1. Relaciones (FK, FK, M2M), choices y Meta en Session.

Conceptos Django demostrados. Se aplican **ForeignKey** (uno-a-muchos), **ManyToManyField** (muchos-a-muchos) y **OneToOneField** (uno-a-uno, en **SpeakerProfile**). **on_delete=PROTECT** evita borrar una sala con sesiones, mientras **CASCADE** propaga el borrado. **related_name** habilita el acceso inverso (**event.sessions.all()**). El detalle completo de tipos de campo está en el Anexo A.

4. Recorrido por la aplicación

Esta sección recorre las pantallas de Tinkuy. En cada una, inserta tu captura en el marco indicado; debajo se explican los conceptos de Django que esa pantalla pone en práctica.

4.1 Inicio — listado de eventos



Figura 2. Listado de eventos publicados.

La vista **event_list** pide los datos al repositorio y renderiza una plantilla:

```
def event_list(request: HttpRequest) -> HttpResponse:
    events = event_repository.list_published() # QuerySet filtrado
    return render(request, "events/event_list.html", {"events": events})
```

Conceptos Django demostrados. Es una **vista basada en función (FBV)** que recibe un **HttpRequest** y devuelve un **HttpResponse** con **render**. Obtiene los datos por la **capa de repositorios** (**Event.objects.filter(is_published=True)**), un **QuerySet**. La plantilla extiende **base.html** y compone la página con **componentes** (**page_header**, **card**, **empty_state**) recorriendo los eventos con **{% for %}**.

4.2 Detalle de evento

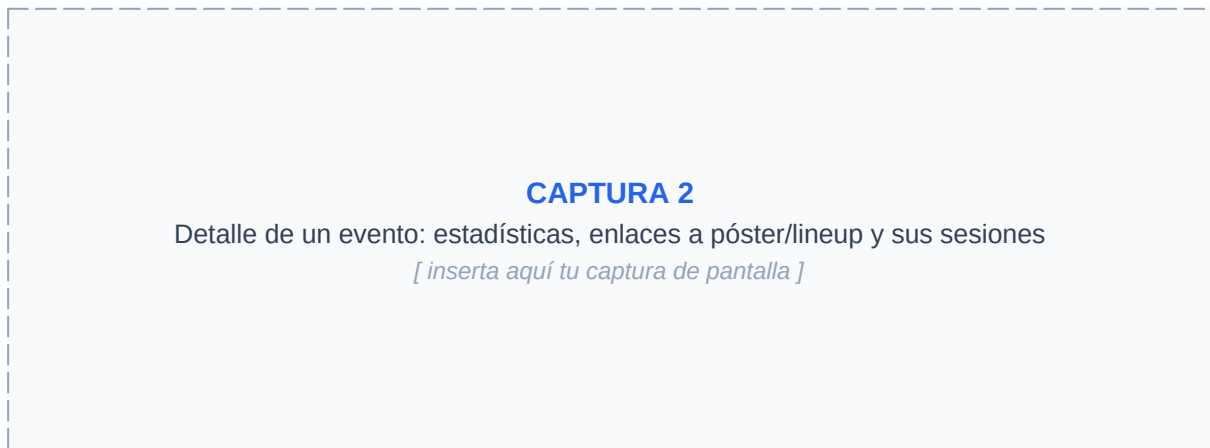


Figura 3. Página de detalle de un evento.

Conceptos Django demostrados. La ruta `eventos/<slug:slug>/` captura el **slug** y la vista resuelve el evento, lanzando `Http404` si no existe. La plantilla genera enlaces con `{% url 'events:event_poster' event.slug %}` (rutas con nombre) y recorre la **relación inversa** `event.sessions.all`. Los datos se muestran con los componentes `stat` y `badge`.

4.3 Sesiones — listado y detalle



Figura 4. Sesiones de un evento.



Figura 5. Detalle de una sesión.

El repositorio optimiza la consulta para evitar el problema **N+1**:

```
Session.objects.filter(event_id=event_id)
    .select_related("room") # JOIN para la FK
    .prefetch_related(Prefetch("speakers", ...)) # 2ª consulta para la M2M
    .order_by("scheduled_at")
```

Conceptos Django demostrados. La sesión demuestra **ForeignKey** (a **Event** y **Room**), **ManyToManyField** (a **Speaker**) y **choices** para el nivel. **select_related** y **prefetch_related** optimizan el acceso a las relaciones. La ruta combina **<int:event_id>** y **<slug:slug>**.

4.4 Ponentes — listado y perfil

CAPTURA 5

Listado de ponentes (nombre, correo, calificación)

[inserta aquí tu captura de pantalla]

Figura 6. Listado de ponentes.

CAPTURA 6

Detalle de un ponente con su perfil (empresa, años de experiencia)

[inserta aquí tu captura de pantalla]

Figura 7. Detalle de ponente y su perfil.

Conceptos Django demostrados. Cada **Speaker** se enlaza con un **SpeakerProfile** mediante **OneToOneField** (**related_name="profile"**), que amplía sus datos sin duplicar el modelo. Se accede con **speaker.profile**. Es la relación **uno-a-uno** del proyecto.

4.5 Salas

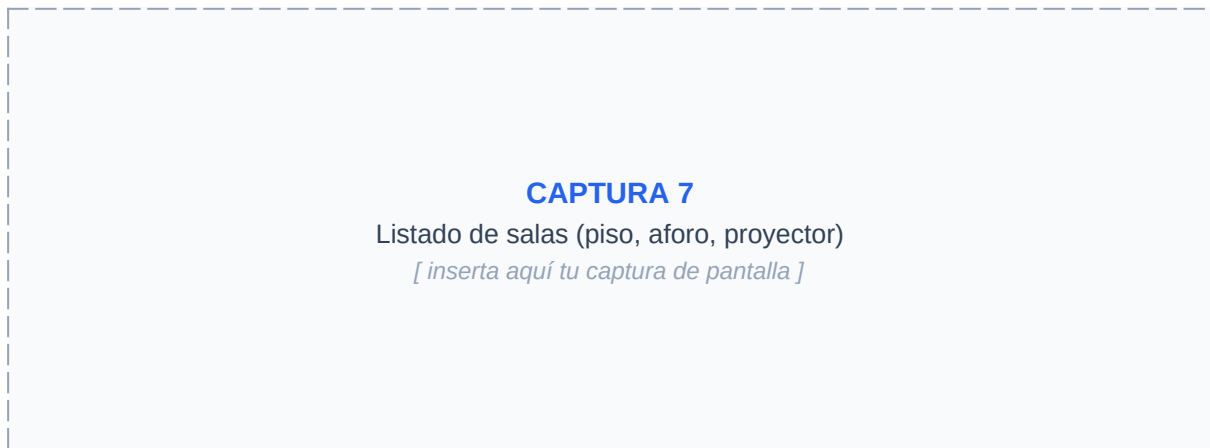


Figura 8. Listado de salas.

Conceptos Django demostrados. La sala usa campos `IntegerField`, `PositiveIntegerField` y `BooleanField`. Su relación inversa `room.sessions` lista las charlas programadas, y `on_delete=PROTECT` en `Session.room` impide borrar una sala en uso.

4.6 Asistentes



Figura 9. Listado de asistentes.

Conceptos Django demostrados. El asistente aplica `EmailField(unique=True)`, `BooleanField` y `registered_on = DateField(auto_now_add=True)`, que registra automáticamente la fecha de alta.

4.7 Registros y reporte de salas

CAPTURA 9

Listado de registros / reporte de utilización de salas

[inserta aquí tu captura de pantalla]

Figura 10. Inscripciones y reporte de utilización.

El reporte de utilización se calcula con **SQL crudo parametrizado** y se mapea a un DTO inmutable:

```
SELECT r.id, r.name, r.floor,
       COUNT(s.id) AS session_count,
       COALESCE(SUM(EXTRACT(EPOCH FROM s.duration) / 60), 0)::int AS total_minutes
FROM events_room r
LEFT JOIN events_session s ON s.room_id = r.id
GROUP BY r.id, r.name, r.floor
ORDER BY total_minutes DESC
```

Conceptos Django demostrados. `Registration` une `Attendee` y `Session` con dos `ForeignKey` y `unique_together` (una inscripción por par). El reporte usa **SQL crudo** con marcadores `%s` (nunca f-strings: evita la **inyección SQL**) y devuelve una `dataclass(frozen=True)`, de modo que el tipo queda fijado en la frontera.

4.8 Pósteres de difusión

CAPTURA 10

Póster del evento (portada, datos y ponentes)

[inserta aquí tu captura de pantalla]

Figura 11. Póster de evento.

CAPTURA 11

Póster de cartelera (lineup) con las charlas destacadas

[inserta aquí tu captura de pantalla]

Figura 12. Póster de cartelera.

Conceptos Django demostrados. Las vistas `event_poster` y `lineup_poster` leen por repositorios e **integran todas las capas**. Las plantillas heredan de un `poster_base.html` con una hoja de estilos de **impresión** (`@media print`) y un motivo andino, manteniendo la identidad del producto. Se sirven como páginas web.

5. Panel de administración

Django genera un **sitio de administración** completo a partir de los modelos. En `events/admin.py` se registran los **ocho modelos** con clases `ModelAdmin` personalizadas (columnas, filtros, búsqueda) e **inlines** para las relaciones.

CAPTURA 12

Índice del admin mostrando los modelos registrados (Eventos, Salas, Ponentes, Sesiones, Asistentes, Registros...)

[inserta aquí tu captura de pantalla]

Figura 13. Índice del panel de administración.

CAPTURA 13

Formulario de Sesión: selector de ponentes (M2M) e inline de Registros

[inserta aquí tu captura de pantalla]

Figura 14. Edición de una sesión con inline y M2M.

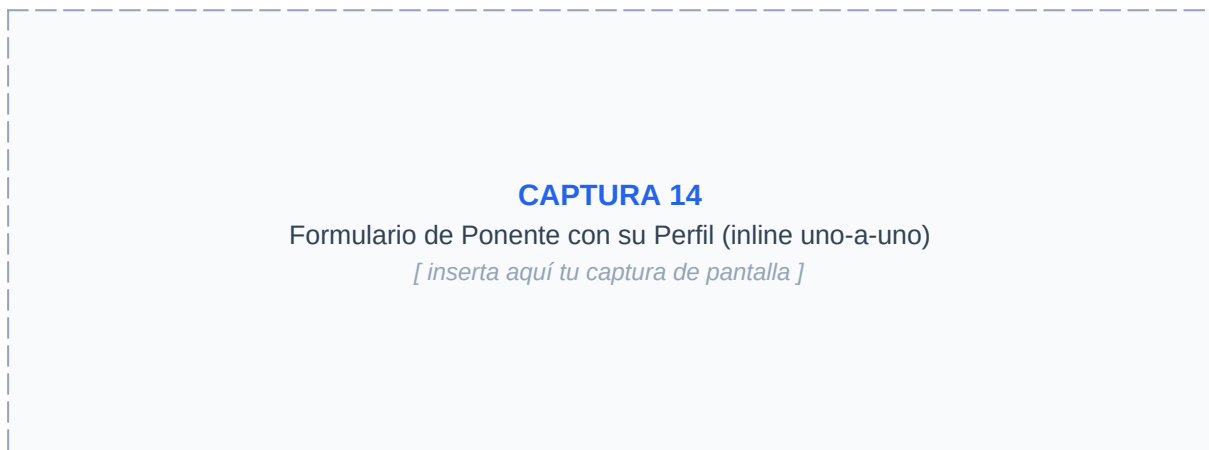


Figura 15. Ponente con perfil en inline.

```
@admin.register(Session)
class SessionAdmin(admin.ModelAdmin):
    list_display = ("title", "event", "room", "scheduled_at", "is_keynote")
    list_filter = ("level", "is_keynote", "event")
    search_fields = ("title", "abstract")
    prepopulated_fields = {"slug": ("title",)}
    filter_horizontal = ("speakers",)          # M2M
    autocomplete_fields = ("event", "room")
    inlines = [RegistrationInline]             # tabla intermedia
```

Código 2. Administración del modelo Session.

Conceptos Django demostrados. Se demuestra el **admin autogenerado** con `@admin.register` y `ModelAdmin`: `list_display`, `list_filter`, `search_fields`, `prepopulated_fields`. Los **inlines** `StackedInline` (perfil 1:1) y `TabularInline` (inscripciones) editan registros relacionados en la misma página; `filter_horizontal` gestiona la M2M y `autocomplete_fields` agiliza las FK. El acceso usa la **autenticación** de Django (`createsuperuser`).

6. Capa de datos y calidad

Toda consulta pasa por `events/repositories/`. Las lecturas simples y escrituras usan el **ORM** (devolviendo `QuerySet` o instancias); los reportes con JOIN y agregados usan **SQL crudo parametrizado** mapeado a DTOs. Esta separación mantiene las vistas y plantillas libres de lógica de datos.

Además, el proyecto mantiene **tipado estricto** con `pyright` y pruebas con `pytest`, una «verja» de calidad que se ejecuta antes de cerrar cada cambio.

7. Conclusión

Tinkuy demuestra, de forma integrada, los pilares de Django: **modelos** con todos los tipos de campo y las tres relaciones; **migraciones** que llevan ese diseño a PostgreSQL; **vistas** por cada modelo que leen a través de **repositorios**; **plantillas** reutilizables compuestas por componentes; y un **panel de administración** para los ocho modelos. Las capturas de las secciones anteriores evidencian cada una de estas piezas en funcionamiento.

Anexo A — Cobertura de tipos de campo

Tipo de campo	Dónde aparece
CharField	Event.name, Room.name, Speaker.full_name, Session.title, Attendee.full_name
TextField	Event.summary, Room.notes, Speaker.bio, Session.abstract
SlugField	Event.slug, Session.slug
EmailField	Speaker.email, Attendee.email
URLField	Event.website, Speaker.twitter_url, Session.recording_url
IntegerField / PositiveIntegerField	Room.floor / Event.capacity, Session.max_seats
PositiveSmallIntegerField	SpeakerProfile.years_experience
DecimalField	Event.ticket_price, Registration.amount_paid
FloatField	Speaker.rating
BooleanField	Event.is_published, Room.has_projector, Session.is_keynote, Attendee.is_student
DateField / DateTimeField / TimeField	Event.start_date / Session.scheduled_at / Session.start_time
DurationField	Session.duration
ImageField / FileField	Event.banner, Speaker.photo / Event.program_pdf
JSONField	SiteConfiguration.value
choices (TextChoices)	Session.level
ForeignKey	Session.event, Session.room, Registration.attendee, Registration.session
ManyToManyField	Session.speakers
OneToOneField	SpeakerProfile.speaker

Tabla 4. Tipos de campo aplicados en los modelos.

Anexo B — Lista de capturas a incluir

Marcos a rellenar con tus capturas (en orden):

- Captura 1 — Inicio: listado de eventos.
- Captura 2 — Detalle de un evento.
- Capturas 3 y 4 — Listado y detalle de sesiones.
- Capturas 5 y 6 — Listado de ponentes y detalle con perfil.
- Captura 7 — Listado de salas.
- Captura 8 — Listado de asistentes.
- Captura 9 — Registros / reporte de salas.
- Capturas 10 y 11 — Póster de evento y de cartelera.
- Capturas 12 a 14 — Admin: índice, sesión (inline + M2M) y ponente (perfil).
- **Captura final** — Último commit subido a GitHub (la añades tú).

Anexo C — Glosario

Término	Definición
ORM	Traduce clases y objetos de Python a tablas y filas.
QuerySet	Consulta perezosa; se evalúa al recorrerla.
Vista (FBV)	Función que recibe HttpRequest y devuelve HttpResponse.
Repositorio	Único módulo que accede a la base de datos.
Inclusion tag	Etiqueta que renderiza un componente con props.
Inline (admin)	Edición de registros relacionados en la página del padre.
DTO	Dataclass congelada con tipos explícitos en la frontera de datos.
N+1	Antipatrón de consultas resuelto con <code>select_related/prefetch_related</code> .

Tabla 5. Glosario.

Anexo D — Último commit en el repositorio

Captura del último cambio subido al repositorio github.com/czambrano1997/ProgII_UTE202601 (rama `estudiante/Narvaez_Santiago`).



Figura 16. Último commit: «project cleanup» — 3d51b6d8 — Santiago Narváez — 20 jun 2026.